Using CNN to classify the Surface of Mars

1.0 Introduction

Convolutional Neural Networks (CNN) are a class of machine learning models designed for analysing image data by learning spatial patterns and features though convolutional filters. Unlike traditional neural networks which rely on prepared and cleaned features, CNNs extract relevant features directly from pixel data using convolutional filters. In this project, CNN is used to analyse images of the martian surface and classify them into 8 terrain categories by learning and distinguishing visual characteristics. During model development, 11 different configurations were tested, with the architecture and parameters reported on this paper corresponding to the best performing model.

2.0 Dataset

The dataset used consists of 73,031 labelled images from HiRISE images of the Martian surface [1]. Each image provided is grayscaled with resolution of 277x277 pixels and is assigned to one of eight terrain categories: other, crater, dune dark, slope streak, bright dune, impact ejecta, swiss cheese and spider.

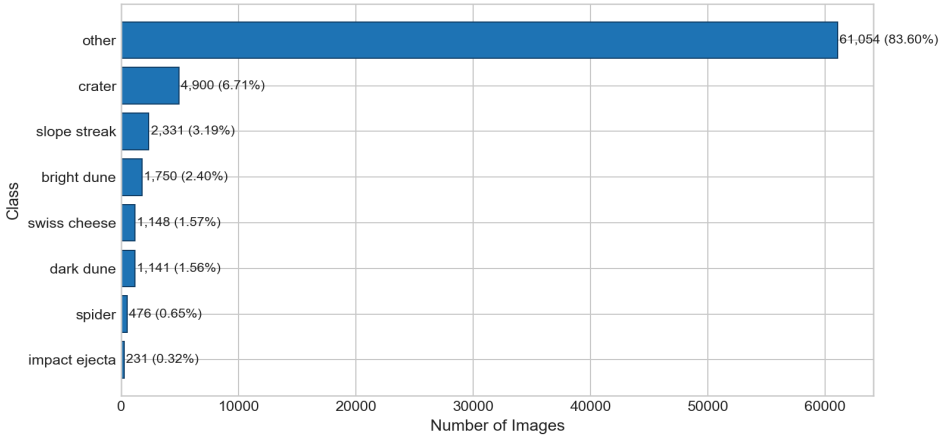


Fig 1. Class Distribution on HiRISE dataset

One issue the dataset has is that the class distribution is highly imbalanced, with the majority of images belonging to the ‘other’ category. As shown in Figure 1, the ‘other’ class accounts for 83.6% of the dataset, which can bias the model towards predicting this category more frequently. This imbalance is particularly significant for the ‘spider’ and ‘impact ejecta’ classes, which together account for less than 1% of the dataset. Since these classes contain very few training examples, the model may struggle to learn their distinguishing features and may misclassify them as the more common classes.

2.1 Dataset Preprosessing

Before training , the images undergo preprocessing to prepare them for input into the CNN, Each image is read, decoded and converted to grayscale before being resized to 160x160 pixels. Different image sizes were tested but this size was selected as it provided a balance

between computational efficiency and preserving enough information for feature recognition. Next, the images' pixel values are normalised to the range [0,1] to ensure numerical stability during training.

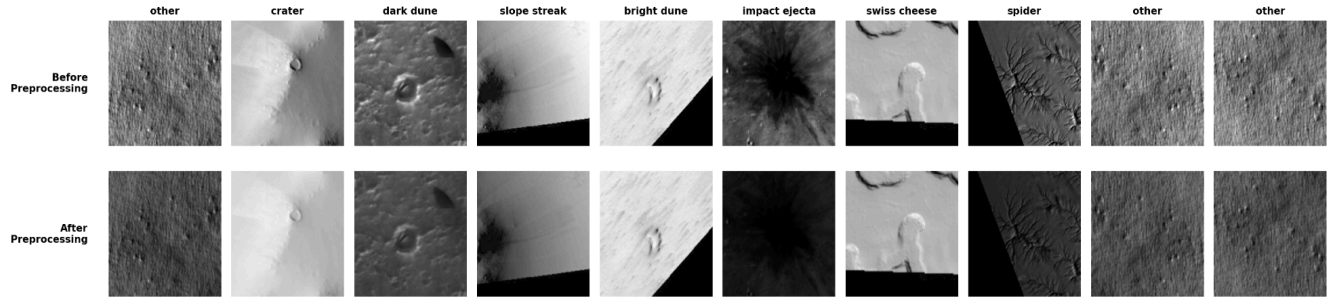


Fig 2. Images before and after preprocessing

2.2 Dataset Split

After preprocessing, the dataset is split into training, validation and test using a 70-10-20 split, with stratified sampling to preserve the original class distribution across all subsets. Finally, during training the dataset is shuffled and grouped into mini-batches of 64 images, which are then fed sequentially to the network.

3.0 Model

The model accepts processed images as input into the network in the form 160x160 normalised grayscale images. The architecture consists of three convolutional layers with 32,64 and 128 3x3 filters respectively, this allows the network to progressively learn increasingly complex spatial features from the images. Each convolutional layer is followed by batch normalisation to further aid numerical stability. After, we used the ReLU activation function which introduces non-linearity into the model while avoiding the vanishing gradient problem. Max-pooling is used after the first two convolutional blocks to reduce spatial dimensions and therefore computational overhead. The resulting feature maps are flattened and passed through a fully connected (dense) layer of 128 neurons, followed by batch normalisation, ReLU and a dropout layer with rate 0.3 to improve generalisation. The final layer is a softmax classification layer with 8 output neurons, each representing one class.

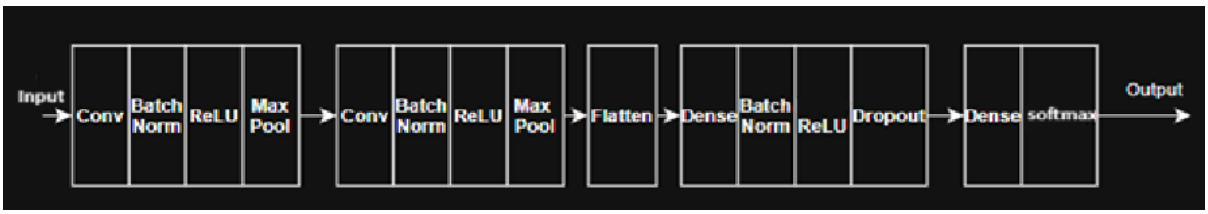


Fig 3. Model Architecture

The model uses He normal weight initialisation for convolutional and dense layers and Glorot uniform initialisation for the output layer, while L2 regularisation with λ=5e-10 is applied to reduce overfitting. For the loss function, focal loss was tested as a possible candidate in an

attempt to combat the strong class imbalance, however, sparse categorical cross-entropy combined with class weights provided more stable performance.

4.0 Training and Testing

The CNN was trained for up to 100 epochs using a batch size of 64. Optimisation was performed using the Adam optimizer with learning rate 0.0005 to introduce adaptive momentum to our model to speed up convergence. To further improve convergence, ReduceLrOnPlateau was implemented. If validation loss did not improve in 1 epoch the learning rate was reduced by 0.75, allowing the model to make smaller optimisation steps as training progressed. Furthermore, early stopping was applied based on validation loss with patience of 10 epochs to prevent unnecessary training once the model stopped improving, while still providing sufficient time for the weights to recover from short-term plateaus. The best performing model weights were automatically restored to be used in testing. In order to improve efficiency and reproducibility, all relevant inputs and outputs of training and testing were saved in the results directory after each run. These included the model parameters, training history, classification report, confusion matrix, test results and model architecture, allowing for full replayability of experiments and easier comparison between model configurations and detailed analysis of model performance.

5.0 Results

In total, 11 different model configurations were developed, with each new model being an improved version. Model performance was evaluated on macro-F1 as it provides a balanced measure of performance across all classes, treating each terrain category equally regardless of dataset imbalance. Nevertheless,other performance metrics were also taken into account if no difference in macro-f1 was observed. The results of the best performing model are shown graphically in Figure 4.

5.1 Train results

The model was trained for a maximum of 100 epochs, however training stopped at epoch 43 due to early stopping as validation loss stopped improving. The loss and accuracy over train and validation subsets is shown in Figure 4.

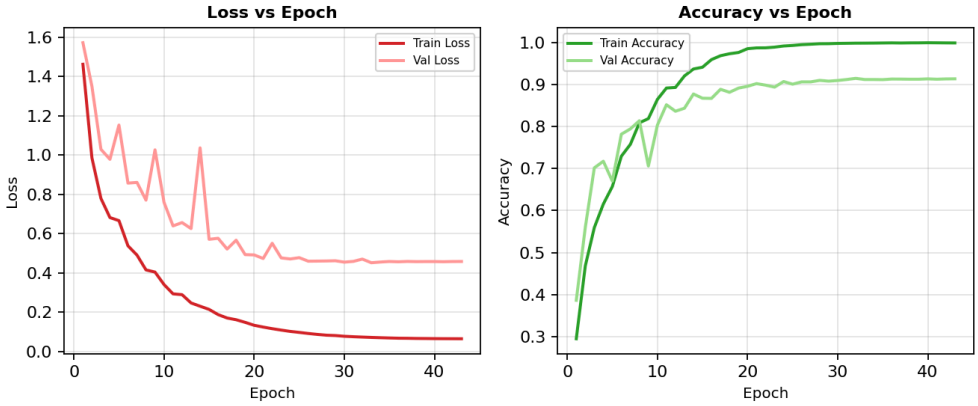


Fig 4. Train and validation loss and accuracy plots over the epochs

The training plots shown in Figure 4 outline the learning behaviour of the model over time. Training loss decreases sharply during the first 10 epochs before gradually plateauing as the model converges, while validation loss also declines initially before stabilising after epoch 25-30. Training accuracy increases rapidly in the first epochs similar to loss until it approaches almost perfect performance(~100%). However, validation accuracy improves and stabilises at around 91%. This gap between training and validation both in loss and accuracy indicates mild overfitting, but the relatively stable validation metrics suggest that the model generalises reasonably well to unseen data.

5.2 Test results

The final model was evaluated on the test set of 14,607 images achieving an accuracy of 91.51% with a loss of 0.438. The model scored a macro F1 score of 0.72 and weighted F1-score of 0.91. These metrics indicate strong overall performance but slightly reduced accuracy on minority classes due to dataset imbalance.

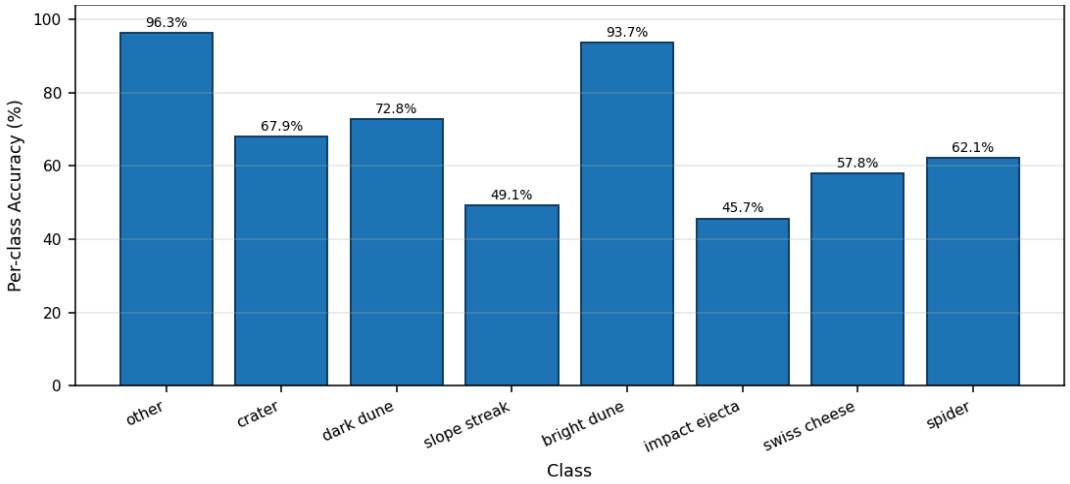


Fig 5. Per Class model accuracy

As shown in Figure 5, the best performance is on the dominant “other” class with 96.3% accuracy, which makes sense since the majority of images in the dataset belong to this class. Accuracy varies in the remaining classes, the “bright dune” class scored 93.7% despite having less images than class “crater” which scored accuracy of 67.9%. Performance is lower for classes like “slope streak” (49.1%) and “impact ejecta” (45.7%). Overall, apart from the dominant “other” class, a higher number of images does not necessarily result in better classification performance. This may be because some terrain types have more visually distinct patterns, therefore making them easier for the model to identify.

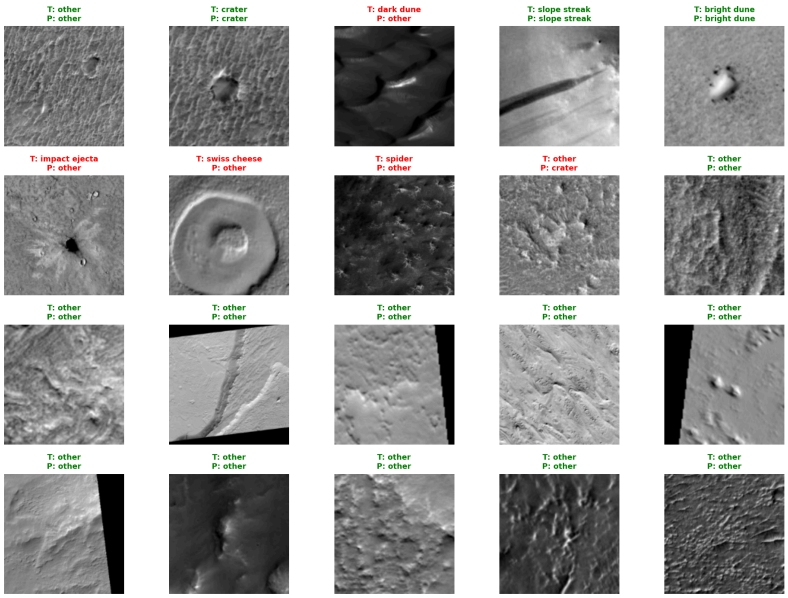


Fig 6. Test Dataset images with labels and predictions

6.0 Conclusion

This project demonstrated that a convolutional neural network can effectively classify Martian terrain. On the other hand, the development of this project faced three main issues. Firstly the computational resources available needed many hours before one training run to complete, cpu parallelism and image downsampling was employed to mitigate the problem. Secondly, the class imbalance present in the dataset is extreme, which biases the model towards predicting the dominant class almost always. This was addressed by applying class weights so the model places greater importance on minority classes, nevertheless as demonstrated in Figure 5, the model still struggled to predict some classes. Thirdly, during the early model development stage the model often showed signs of overfitting, as training accuracy approached near-perfect performance while validation stopped improving at a lower level. To combat this, techniques such as dropout, L2 regularisation were implemented to improve generalisation. Overall, despite these challenges, the final model achieved strong performance metrics and demonstrated that CNNs can be applied to planetary surface analysis.

References

1. NASA, 2022. Mars Orbital Image HiRISE Labeled Data Set Version 3. Available at: https://data.nasa.gov/dataset/mars-orbital-image-hirise-labeled-data-set-version-3/resource/c93bf426-1eae-4d3b-8afd-548add5e24ce [Accessed 6 March 2026].